Brief del proyecto: TaskNest · Onboarding Guiado

Qué es: la resolución completa del caso de Data Analyst de TaskNest. Incluye el análisis, un dashboard interactivo, dos decks, una skill para replicar el análisis, una versión 100 % en Google Sheets y el despliegue público. **Pregunta central:** ¿el CPO debe expandir, modificar o detener el Onboarding Guiado? **Respuesta: expandirlo al 90 % de los usuarios nuevos y dejar un 10 % de holdout aleatorio.** La caída del dashboard no la causa Guided.

Entregable	Dónde
Sitio público (landing con las 4 opciones)	https://tasknest-business-case.vercel.app
Análisis en Google Sheets	https://docs.google.com/spreadsheets/d/1093RwUhI0yNnJEYubFMV7DY_s0dMPBRGRhFg
Deck de análisis (PDF, 8 slides, inglés)	`entregables/TaskNest_Analysis_Deck_EN.pdf`
Deck de uso de IA (PDF, 7 slides, inglés)	`entregables/TaskNest_AI_Use_Deck_EN.pdf`
Reporte de análisis completo (español)	`entregables/ANALISIS_COMPLETO_TASKNEST.md`
Skill para replicar el análisis	`.claude/skills/tasknest-analyst/`

Paso 1 · Entender el problema de negocio (con IA)

Qué: entender el caso, los datos disponibles y los supuestos previos antes de tocar un solo número.

Cómo:

1. Leí el brief del caso (`TaskNest_Candidate_Brief_es.md`) y los tres documentos de trabajo previos:
   - `FASE_1_COMPRENSION_Y_SUPUESTOS.md` (supuestos de negocio);
   - `DATA_UNDERSTANDING_ENGINEERING_BRIEF.md` (supuestos de datos);
   - `GUIA_BUSINESS_CASE_TASKNEST.md` (borrador de la guía y la recomendación).
2. Revisé tu skill **`meli-bi-analyst`** y adopté su forma de trabajar, no sus tablas de MELI:
   - verificar el schema antes de escribir queries;
   - separar la variación de un ratio en «tasa» y «mix» (Paasche + mix centrado en el periodo base), con verificación de cierre obligatoria;
   - agregar numerador y denominador antes de dividir;
   - el pipeline de 5 fases (KPI → funnel → ranking de drivers → eventos → recomendación);
   - hablar de correlación, nunca de causalidad.

3. Ubiqué los tres eventos que coinciden en el tiempo y que hay que separar:
    - el lanzamiento de Guided (1-jun, sin aleatorizar);
    - la Creator Campaign (1-jun);
    - Android 3.2, que cambió cómo se registran las tareas (20-jun).

Resultado: el marco del análisis. La pregunta real no es «¿bajaron los KPIs?», sino **«¿cuánto de la caída le corresponde a cada uno de los tres cambios?»**.

Paso 2 · Subir la data a PostgreSQL

Qué: una base relacional donde cada número sea una consulta SQL reproducible y escalable.

Cómo (infraestructura ya existente en el proyecto):

- `docker-compose.yml`: PostgreSQL 16 en Docker (contenedor `tasknest-postgres`, base `tasknest`).
- `sql/01_schema.sql`: tablas `trial_users` (1,800 filas) y `product_events` (10,220 filas), con índices.
- `sql/02_load_data.sql`: carga de `trial_users.csv` y `product_events.csv`.
- `sql/03_analytical_views.sql`: vistas `v_user_analytics`, `v_executive_dashboard`, `v_simpson_breakdown` y `v_user_events_7d`.
- `scripts/setup_postgres.sh` y `scripts/verify_db.sh`: levantan y verifican la base.

Por qué PostgreSQL: es escalable (el mismo SQL sirve con 1,800 filas o con volúmenes de producción), es reproducible y es una sola fuente de verdad para todos los entregables.

Paso 3 · Conectar Claude Code a PostgreSQL con MCP

Qué: que la IA consulte los datos en vivo, sin poder modificarlos.

Cómo:

- `.mcp.json` registra el servidor `postgres-tasknest` (`@modelcontextprotocol/server-postgres`), que es **de solo lectura**.
- Todas las consultas del análisis se corrieron con la herramienta `mcp__postgres-tasknest__query`.

Detalle importante que se descubrió después: el cliente del MCP **muestra los timestamps desplazados** por el huso horario local; los cálculos dentro de SQL sí son correctos. Esto quedó documentado y se corrigió donde afectaba (ver Paso 8).

Paso 4 · Revisar los supuestos de la data

Qué: auditar los datos y los documentos previos antes de concluir nada.

Cómo:

1. Recalculé todo **desde las tablas crudas**, sin confiar en las vistas, y revisé el código de cada vista línea por línea.
2. Revisé la integridad: duplicados, eventos huérfanos, versiones inconsistentes y rangos de fechas.
3. Encontré **4 trampas en los datos**:

Trampa	Evidencia	Tratamiento
Android 3.2 deja de registrar tareas	Tareas por usuario 2.81 → 0.94; usuarios con proyecto sin cambio	La activación de 3.2 se trata como no confiable; su conversión sí sirve
Desfase de reloj	1,928 eventos con hora anterior al signup (hasta −15 h)	Versión de sensibilidad con tolerancia de −15 h
Cohortes recientes sin 14 días completos	Registros del final de julio	Resultados repetidos solo con cohortes completas
Pagos justo después del día 14	3 de 552 (el cobro corre en un proceso por hora)	Regla oficial de ≤14 días; incluirlos no cambia nada

4. Encontré un **sesgo oculto en la vista oficial de activación**: como descarta eventos previos al signup, los usuarios que se registran de noche aparecen 13 pp menos activados. Es un artefacto; corregido, la activación es pareja a cualquier hora.
5. Encontré errores en los borradores previos:
   - ingreso por trial Pre de $4.85 en lugar de $4.82;
   - conteos por precio 434/970/396 en lugar de 450/986/364;
   - «Guided gana en el 100 % de los segmentos» (3 de 12 cruces son negativos);
   - «Guided reduce la sensibilidad al precio» (no se sostiene);
   - «expandir al 100 % de inmediato» (la evidencia es observacional).

Paso 5 · Análisis completo (negocio + datos + IA, guiado por mí)

Qué: responder la pregunta del CPO y ir tres pasos más allá para entender a los usuarios. **Dónde:** `ANALISIS_COMPLETO_TASKNEST.md`.

Cómo (unas 25 consultas SQL vía MCP):

1. **La caída de conversión es mix.** La descomposición tasa/mix por canal da tasa +3.79 pp y mix −7.49 pp, con total −3.70 pp y **cierre exacto** . Los 4 canales mejoraron; la Creator Campaign pasó de 3.7 % a 34.1 % de los registros y convierte a ~13 %. Con el mix de canales de abril-mayo, junio-julio habría convertido **36.8 %**.
2. **La caída de activación es un bug de medición** en Android 3.2: los usuarios con ≥3 tareas caen de 51 % a 8 %, pero los proyectos y la conversión no se mueven.

3. **Guided contra Classic, comparados en las mismas semanas:**
   - conversión 31.7 % contra 24.6 %: **+7.2 pp, IC 95 % [+1.6, +12.7]**;
   - activación sin 3.2: +9.1 pp;
   - ingreso por trial: +31 %.
4. **Pruebas de robustez:**
   - controlando por canal × plataforma, el lift queda en +6.9 pp;
   - Classic no cambia antes y después del lanzamiento dentro de cada canal (no hay tendencia oculta);
   - Guided gana en las 4 plataformas;
   - con cohortes completas el lift es +8.7 pp.
5. **Lo que se aprendió de los usuarios:**
   - el **aha moment es la 3ª tarea**: la conversión pasa de ~17 % a ~41 %;
   - el **precio casi no mueve la conversión** (elasticidad −0.14), así que $18 deja +42 % de ingreso por trial frente a $12;
   - el **74 % paga en los primeros 7 días**;
   - **Referral vale 3.6 veces más** por trial que la Creator Campaign.
6. **Lo que se descartó** por ser artefacto o ruido: la «activación nocturna» baja, el «jueves» que convierte más, el efecto de invitar colaboradores, y la subida de conversión de Android con 3.2.

Decisiones que tomé yo:

- cuál era la pregunta real;
- qué cuenta como comparación justa (mismas semanas, no periodos);
- tratar Android 3.2 como problema de medición y no de comportamiento;
- qué método usar;
- qué tan fuerte puede ser la afirmación (observacional → recomendar un holdout).

Paso 6 · Dashboard interactivo

Qué: una app para explorar la conversión y los KPIs de tareas y proyectos, con filtros y comparativos.
Dónde: `app/` (React 19 + Vite + Tailwind 4 + TypeScript, el mismo stack de tu app `clara`).

Cómo:

1. **Datos:** `sql/04_dashboard_export.sql` exporta con SQL una fila por usuario con todas las banderas calculadas (`scripts/export_dashboard_data.sh` → `app/public/data/users.json`). La app es estática: el navegador filtra y agrega.
2. **Diseño:** seguí la skill de visualización:
   - paleta validada con su script (daltonismo, modo claro y oscuro);
   - tooltip con crosshair, leyenda con etiquetas directas, vista de tabla;
   - puntos huecos cuando el dato es frágil.
3. **Iteraciones contigo:**

- **v1:** Timeline + Comparativo con filtros por canal, dispositivo, experiencia, versión y precio.
- **v2:** rango de fechas libre, filtros Incluir/Excluir y segmentos personalizados (ej. «Creator · iOS» contra «Creator · Android»).
- **v3 (más intuitiva):** filtros escritos como frases («Canal **no es** Creator Campaign»), un solo concepto de «grupos» con pasos numerados, 8 preguntas guía con su respuesta, y la regla técnica movida a Opciones.

4. **Avisos que evitan malas lecturas:**
   - Android 3.2 en los KPIs basados en tareas;
   - cohortes incompletas;
   - grupos chicos;
   - comparar experiencias con fechas previas al 1-jun.

Validación: pruebas con navegador automatizado (Playwright) en escritorio, móvil y modo oscuro. Los números del dashboard coinciden con el SQL en cada escenario probado.

Paso 7 · Deck de análisis

Qué: la historia para el CPO, en inglés. **Dónde:** pestaña «Deck de análisis» de la app y `Task–Nest_Analysis_Deck_EN.pdf`.

Cómo:

1. Primera versión de 19 slides con todo el storytelling: método, auditoría, mix, robustez, usuarios, lo descartado e impacto.
2. Por tu feedback, la reduje a **8 slides**, sin «Step X», con títulos que ya dicen la conclusión y una gráfica principal por slide:
   - la respuesta;
   - los tres cambios simultáneos;
   - la caída de conversión como mix (cascada + antes/después por canal);
   - la caída de activación como bug de Android 3.2;
   - Guided gana (barras con IC + franja de robustez);
   - lo que aprendimos de los usuarios;
   - la recomendación;
   - el apéndice.
3. Las gráficas son SVG propias que siguen la skill de visualización. Tiene notas para presentar (tecla N) y un modo de impresión; el PDF se genera con Playwright.

Validación: revisé visualmente cada slide, con una prueba automática que confirma que nada se desborda. Las cifras se reconciliaron con el reporte y el dashboard.

Paso 8 · Skill para replicar el análisis

Qué: `tasknest-analyst`, una skill en inglés con la misma arquitectura que `meli-bi-analyst`. Con ella, un stakeholder con Claude Code y el MCP puede hacer preguntas nuevas o revisar data nueva. **Dónde:** `.claude/skills/tasknest-analyst/` (`SKILL.md` + 8 referencias).

Cómo:

- **Contenido:**
  - `SKILL.md`: rol, conexión, tabla de ruteo por tipo de pregunta, marco analítico, checklist y ejemplos.
  - `metrics.md`: definiciones canónicas + el CTE `user_facts`, del que salen todas las métricas.
  - `queries.md`: 12 plantillas SQL.
  - `methodology.md`: tasa/mix, comparación con IC y lectura del holdout cuando llegue.
  - `new_data_playbook.md`: qué hacer cuando llega data nueva.
  - `business_context.md`: cifras base y umbrales que cambiarían la recomendación.
  - `pitfalls.md` y `report_format.md`.
- **Validación:**
  1. Un script ejecutó los **18 bloques SQL** de la skill contra la base, y todos reproducen el análisis.
  2. **Simulé la llegada de data nueva** (un canal nuevo y Android 3.3) dentro de una transacción con `ROLLBACK`. El playbook detectó ambos cambios, y los datos reales quedaron intactos.
- **Correcciones que salieron de esta prueba:**
  - la hora de extracción estaba mal documentada (era la versión desplazada por el MCP);
  - había 3 definiciones distintas de «cohorte completa», unificadas en una regla conservadora;
  - el histograma de días a pago salía de una vista con redondeo.

Paso 9 · Deck de uso de IA

Qué: la nota de uso de IA que pide el brief, como deck de 7 slides en inglés. **Dónde:** pestaña «Deck de uso de IA» de la app y `TaskNest_AI_Use_Deck_EN.pdf`.

Contenido: tu pipeline de 7 pasos con quién lideró cada uno:

1. entender el negocio (juntos);
2. PostgreSQL (lo hizo la IA, revisado por ti);
3. MCP (lo hizo la IA, revisado por ti);
4. revisión de supuestos (lideraste tú, la IA ejecutó);
5. análisis guiado (lideraste tú, la IA ejecutó);
6. entregables (los hizo la IA, revisados por ti);
7. skill (juntos).

También muestra la arquitectura de datos, los 9 problemas detectados en borradores hechos con IA, la división «yo decidí / la IA ejecutó», una fuente de verdad con cuatro entregables, y la validación con sus límites.

Para no duplicar código, convertí el visor de slides en un componente reutilizable que comparten los dos decks.

Paso 10 · Análisis rehecho 100 % en Google Sheets

Qué: que un analista que nunca ha usado IA pueda auditar todo con fórmulas y tablas dinámicas, como si se hubiera hecho a mano.

Conexión robusta a tu cuenta personal:

1. **Credencial:** service account de tu proyecto personal de Google Cloud. La llave se movió **fuera del proyecto**, a `~/.config/tasknest-gsheets/service_account.json` (permisos 600). Elegí service account en lugar de OAuth porque el token de OAuth personal vence cada 7 días.
2. **Cliente:** `tools/gsheets/gsheets_client.py` usa la API oficial de Sheets v4, con reintentos ante límites de cuota de Google y errores temporales, y escritura por lotes.
3. **MCP de Google Sheets:** registrado en `.mcp.json`. La versión publicada (`mcp-google-sheets` 0.6.3) no arrancaba con el SDK de MCP 2.x; se resolvió fijando `mcp<2`. Lo probé con un handshake MCP real: 20 herramientas, y lectura de tu Sheet.

Construcción: `tools/gsheets/build_workbook.py` reconstruye el libro desde cero en ~40 s y se puede volver a correr sin problema.

Capa	Hojas	Contenido
Datos	`raw_trial_users`, `raw_product_events`	CSV pegados tal cual. Es lo único pegado como valores
Supuestos	`params`	Lanzamiento, ventanas de 7 y 14 días, tolerancia, corte. Son rangos con nombre y todo se recalcula si cambian
Preparación	`events_prep`	Una fila por evento: XLOOKUP del signup, minutos desde el signup, banderas de ventana
Modelo	`user_facts`	Una fila por usuario: COUNTIFS → activación, conversión, ingreso, madurez
Exploración	`Pivot Tables`	5 tablas dinámicas nativas
Análisis	1–7	Calidad, dashboard contra realidad, mix (con verificación de cierre y cascada), Guided contra Classic (IC, lift controlado, estabilidad, plataformas), Android 3.2, usuarios y timeline. AVERAGEIFS / COUNTIFS / SUMPRODUCT sobre rangos con nombre, más 8 gráficas nativas
Cierre	`Conclusions`, `README`	Respuesta, hallazgos con evidencia en texto vivo (fórmulas), recomendación, índice y guía para auditar

Validación:

- `verify_workbook.py` compara **61 celdas clave contra el SQL: 61/61** , y encuentra **0 errores de fórmula**.

- Revisé visualmente las hojas exportándolas a PDF.

El Sheet encontró dos redondeos dobles de los entregables anteriores, que se corrigieron en todos lados (reporte, deck, skill y PDF):

- contrafactual de mix: **36.8 %**, no 36.9 %;
- pagadores en 7 días: **74.3 %**, no 74.4 %.

Paso 11 · Despliegue público

Qué: publicar la app para que cualquiera la vea.

Cómo:

1. Antes de subir, revisé el contenido de `dist/`: solo el sitio estático (2 MB) con datos sintéticos, sin llaves ni `.env`.
2. Desplegué en **Vercel**. Quedó en tu cuenta (`jesus105`) como el proyecto `tasknest-business-case`.
3. Cambié el nombre del proyecto (había salido como «dist») y agregué el dominio público `tasknest-business-case.vercel.app`. Un alias suelto quedaba detrás de la protección de Vercel; como dominio del proyecto, es público.
4. Para publicar de nuevo: `npm run deploy` desde `app/`. El script restaura el vínculo con el proyecto, que cada build borraba.

Validación:

- La página, los datos y los PDFs responden 200 sin sesión.
- El Sheet se puede ver públicamente.
- Prueba en vivo con Playwright: landing con 4 tarjetas, dashboard con datos y los dos decks, sin errores.

Controles de calidad que se aplicaron en todo

1. **Solo tablas crudas:** cada número se recalcula desde `trial_users` y `product_events`; las vistas se revisan pero no se dan por buenas.
2. **Verificación de cierre:** las descomposiciones suman exacto al cambio total.
3. **Conciliación entre entregables:** reporte = dashboard = deck = skill = Sheets, número por número.
4. **Pruebas de artefacto, mix y ruido** antes de reportar cualquier hallazgo.
5. **Pruebas automáticas:** 18 bloques SQL de la skill, 61 celdas del Sheet, pruebas de navegador para la app y los decks.
6. **Revisión humana** de cada afirmación y de la recomendación.

Errores encontrados y corregidos durante el proyecto

#	Error	Dónde estaba	Corrección
1	Ingreso por trial Pre $4.85	Guía previa	$4.82
2	Conteos por precio 434/970/396	Brief de datos previo	450/986/364
3	«Guided gana en el 100 % de los segmentos»	Guía previa	3 de 12 cruces negativos (chicos)
4	«Guided reduce la sensibilidad al precio»	Guía previa	El precio es igual de inelástico en Classic
5	«Expandir al 100 %, irrefutable»	Guía previa	Observacional → 90/10 con holdout
6	La vista de activación descarta eventos con desfase de reloj	SQL (vista)	Sesgo de −13 pp en signups nocturnos, documentado
7	Tres definiciones de «cohorte completa»	Reporte, dashboard y skill	Una regla conservadora
8	Histograma de días a pago sobre una vista redondeada	Deck	Recalculado sin redondeo
9	Hora de extracción leída de la salida del MCP	Skill	El MCP desplaza la hora; se corrigió en SQL
10	Contrafactual 36.9 % y 74.4 % (redondeo doble)	Reporte, deck y skill	36.8 % y 74.3 % (lo detectó el Sheet)

Mapa de archivos

```
TaskNest_Clara_Repo/
├── LEEME_PRIMERO.md                  ← guía para personas no técnicas
├── README.md                         ← guía técnica
├── entregables/                      ← lo que se presenta
│   ├── TaskNest_CVR_Analysis.xlsx      análisis completo en Excel (todo con fórmulas)
│   ├── TaskNest_Analysis_Deck_EN.pdf   deck de análisis (8 slides)
│   ├── TaskNest_AI_Use_Deck_EN.pdf     deck de uso de IA (7 slides)
│   └── ANALISIS_COMPLETO_TASKNEST.md   reporte completo (español)
├── docs/
│   ├── BRIEF_PROYECTO.md             ← este documento
│   ├── 01_caso/                      ← brief original (es + pdf)
│   ├── 02_comprension/               ← supuestos de negocio y de datos
│   └── 03_borradores_previos/        ← guía inicial (contiene cifras luego corregidas)
├── data/raw/                         ← trial_users / product_events (.csv y .xlsx)
├── docker-compose.yml                ← PostgreSQL 16
├── sql/                              ← 01 schema · 02 carga · 03 vistas · 04 export app
├── scripts/                  ← setup_postgres.sh, verify_db.sh, export_dashboard_data.sh, make_zip.
├── analysis/analisis_tasknest.py     ← script de análisis (versión previa)
├── .mcp.json                         ← MCP postgres-tasknest
├── .claude/skills/tasknest-analyst/  ← skill para replicar el análisis
├── app/                              ← landing, dashboard y decks (React) → Vercel
└── tools/gsheets/                    ← conector y constructor de Google Sheets
```

Documentos previos que se conservaron como contexto: `FASE_1_…`, `GUIA_…`, `DATA_UNDERSTANDING_…` y `analisis_tasknest.py`. **Ojo:** contienen cifras que se corrigieron después (ver la tabla de errores).

Cómo reproducir todo

```
# 1. Base de datos
docker compose up -d                        # PostgreSQL con los CSV cargados

# 2. App (dashboard + decks)
cd app && npm install
npm run data                                # exporta users.json desde Postgres con SQL
npm run dev                                 # http://localhost:5173
npm run deploy                              # publica en https://tasknest-business-case.vercel.app

# 3. Google Sheets
cd tools/gsheets
python3 -m venv .venv && .venv/bin/pip install google-api-python-client google-auth
.venv/bin/python build_workbook.py          # reconstruye el libro (~40 s)
.venv/bin/python verify_workbook.py         # 61 celdas contra SQL + errores de fórmula

# 4. Skill
# Abrir el proyecto en Claude Code (aprobar los MCP de .mcp.json) y preguntar,
# por ejemplo: "¿cómo convirtió Creator Campaign en iOS en julio?"
```

Límites

- Datos sintéticos del caso.
- El rollout no fue aleatorio: la evidencia es observacional fuerte; el holdout daría la prueba causal.
- El ingreso considera solo el primer pago (sin LTV ni churn).
- La IA puede equivocarse con seguridad: todo lo que produjo se trató como borrador hasta verificarlo.